

Dialogue Frame Finder

Project Approach, Design Decisions and Findings

Submitted as part of Quest Software Engineering Assignment

By:

Swaathi B

From Shiv Nadar University Chennai

1. Understanding the Problem

The problem statement says: given a video URL and a target dialogue line, identify the exact frame where that dialogue visually appears on screen. The output must include a timestamp, frame number, recognized text, and the frame saved as an image.

At first glance this might seem straightforward. You could just run speech recognition and call it done. But the problem specifically says visually appears, not spoken. That distinction changed everything about how I approached this.

On-screen captions and spoken dialogue are related but not the same thing:

- Captions often appear a few seconds after the actor speaks the line.
- Some dialogues appear as title cards or text overlays and are never spoken at all.
- Speech recognition gives you an audio timestamp, not a visual frame.

So the real task is: find the frame where a specific string of text is rendered in the video's pixel data. Speech recognition is only useful for narrowing down where to look. The actual detection has to be done by reading pixels, which means OCR.

The video given was hosted on OK.ru. That added another layer of complexity since OK.ru actively blocks programmatic downloads using TLS fingerprint detection.

2. How I Thought About the Solution Before Writing Any Code

Before starting implementation I spent time researching what approaches were actually available and what tradeoffs each one had. I did not just pick the first idea.

The approaches I considered:

Option A: OCR every frame

Simplest to explain, guaranteed to work if implemented correctly. The problem is that a typical video at 25 fps with 10 minutes of content has around 15,000 frames. Running PaddleOCR on each one would take hours. Not practical.

Option B: OCR at a fixed sampling rate (for example every 1 to 2 seconds)

Better. If I sample every 1 second I only process 600 frames for a 10-minute video. The risk is missing a subtitle that appears for less than 1 second, or getting only a rough timestamp without knowing the exact first frame.

Option C: Speech recognition (ASR) only

Fast and cheap. But as I reasoned above, this answers when was it spoken, not when did it appear visually. Rejecting this as a standalone solution was one of the most important decisions I made.

Option D: Subtitle track extraction

OK.ru videos often do not have embedded subtitle tracks. Even if they did, extractable subtitles are rendering instructions, not proof that the text is actually visible on screen at a given frame.

Option E: ASR for the window, then OCR to find the exact frame

This was the approach I chose. The logic is: speech recognition is fast and gives you an approximate time range. OCR is slow but accurate. Use them together. Run ASR first to find a rough window of 10 to 15 seconds, then run OCR only on frames within that window to pinpoint the exact frame.

This turns an O(all frames in video) problem into an O(frames in a small window) problem. That is the core engineering insight behind the pipeline.

The above text is my writing and below table is AI generated through prompt for easy understanding

Approach	Finds Exact Frame?	Computational Cost	Works if Text Not Spoken?	My Decision
OCR every frame	Yes	Very high	Yes	Rejected - too slow
Fixed-rate OCR sampling	Only approximate	Medium	Yes	Used as fallback
ASR only	No - audio only	Low	No	Rejected as primary
Subtitle extraction	No guarantee	Very low	No	Skipped - unavailable
ASR + OCR (chosen)	Yes	Low-medium	Yes (fallback)	Chosen

3. Architecture: Why I Split the Code This Way

I could have put everything in one large script. I deliberately chose not to, and for a specific reason: when the interviewer asks me to modify or explain one part, I need to be able to point to one file and change only that file without breaking everything else.

Each module has exactly one job:

File	Responsibility
downloader.py	Wraps yt-dlp. Downloads a URL to a local video file. Raises DownloadError on any failure.
audio.py	Uses ffmpeg to extract a 16kHz mono WAV from the video for Whisper transcription.
frame_extractor.py	Uses OpenCV to read real FPS, total frames, and extract a frame at a given index.
transcriber.py	Wraps faster-whisper. Returns timestamped text segments from audio.
matcher.py	Normalizes text (lowercase, strip punctuation) and scores similarity using RapidFuzz partial_ratio.
localizer.py	Stage 1: fuzzy-matches ASR segments to find the search window. Stage 2: fixed-interval fallback scan.
frame_search.py	Stage 3: coarse sampling inside window, bisection to find first matching frame, persistence check.
ocr.py	Wraps PaddleOCR. Takes a frame (numpy array), returns list of (text, confidence) detections.
config.py	All tunable constants in one place. No magic numbers anywhere else.
models.py	Dataclasses: MatchResult, Confidence enum, FrameCandidate, PipelineResult.
pipeline.py	Orchestrates all stages end to end for a given video path and dialogue string.
cli.py	argparse command-line entry point.
app.py	Optional Flask web UI for browser-based testing.

All tunable values (match thresholds, sampling intervals, model size, padding window) live in config.py. This was a deliberate decision. For example, If I found out that confidence threshold seems too strict, I can open config.py, change one number, and rerun. Nothing else needs to change.

4. The Pipeline Step by Step

Stage 0: Download

The video URL is passed to yt-dlp. If anything fails, the pipeline raises a DownloadError immediately and returns a structured 'not found' result. No partial results, no guessing.

For OK.ru specifically: yt-dlp cannot download the video directly because OK.ru uses TLS/JA3 fingerprint filtering to block Python's SSL stack. When yt-dlp tries to fetch metadata, OK.ru detects the non-browser TLS handshake and resets the connection. I confirmed this by checking that YouTube worked fine from the same machine, ruling out a general network issue.

The fallback I implemented uses Playwright, which launches a real headless Chromium browser. Because it is an actual browser, the TLS fingerprint is genuine and OK.ru does not block it. Playwright navigates to the OK.ru page, intercepts the CDN stream URL from network requests, and downloads it.

```
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> python -m dialogue_finder
"https://ok.ru/video/248244667877" "My mind rebels at stagnation" --verbose
INFO dialogue_finder.pipeline: Stage 1: Downloading video from https://ok.ru/video/248244667877
ERROR yt_dlp: ERROR: [Odnoklassniki] 248244667877: Unable to download webpage: ('Connection aborted.',
ConnectionResetError(10054, 'An existing connection was forcibly closed by the remote host', None, 10054,
None))
WARNING dialogue_finder.downloader: yt-dlp failed on https://ok.ru/video/248244667877 - trying Playwright
Chromium fallback...
INFO dialogue_finder.downloader: Playwright: Launching Chromium (headless)...
INFO dialogue_finder.downloader: Playwright (headless): Navigating to https://ok.ru/video/248244667877
INFO dialogue_finder.downloader: Playwright: CDN URL captured via response handler.
INFO dialogue_finder.downloader: Playwright (headless): Downloading CDN stream...
INFO dialogue_finder.downloader: Playwright (headless): Download complete: .temp\video.mp4
WARNING dialogue_finder.downloader: NOTE: Playwright fallback often downloads a low-resolution stream
(144p-240p) without audio because OK.ru CDN serves the smallest chunk during initial browser load. For
full quality, download manually.
INFO dialogue_finder.pipeline: Stage 1 complete. Video saved to .temp\video.mp4
INFO dialogue_finder.pipeline: Stage 2: Extracting audio for ASR...
```

Stage 1: ASR Coarse Localization

The video's audio is extracted to a 16kHz mono WAV. This runs through faster-whisper (tiny model, int8 quantization) to produce timestamped text segments.

Each segment is scored against the target dialogue using RapidFuzz partial_ratio on normalized text. The best matching segment defines a search window. I pad the window 2 seconds before and 4 seconds after the segment boundaries to account for caption timing delays.

Why the padding? Because in a Sherlock Holmes video, the character speaks the line, and the on-screen subtitle appears a few seconds later. Without padding I would miss it.

If no ASR segment scores above ASR_MATCH_THRESHOLD (60), the pipeline falls back to a fixed-interval scan of the full video at 1.5-second intervals. Slower, but guaranteed to find it if it is there.

```
INFO:dialogue_finder.pipeline:Video: sherlock.mp4
INFO:dialogue_finder.pipeline:Stage 2: Reading video metadata
INFO:werkzeug:127.0.0.1 - - [26/Aug/2026 22:38:13] "GET /api/stream/3a8b0014-5cdb-4e33-847d-9236e088c12a HTTP
/1.1" 200 -
INFO:dialogue_finder.frame_extractor:Video info: 23.98 fps, 78204 frames, 3261.7 sec, 960x720
INFO:dialogue_finder.pipeline:fps=23.9762 frames=78204 duration=3261.7s
INFO:dialogue_finder.pipeline:Stage 3: Extracting audio
INFO:dialogue_finder.audio:Audio extracted: C:\Users\Admin\AppData\Local\Temp\dff_work_pk9vjd26\audio.wav (10
1930 KB)
INFO:dialogue_finder.pipeline:Stage 4: Transcribing audio with Whisper
INFO:dialogue_finder.transcriber:Loading Whisper model 'tiny' (CPU, int8)...
INFO:httpx:HTTP Request: GET https://huggingface.co/api/models/Systran/faster-whisper-tiny/revision/main "HTT
P/1.1 200 OK"
INFO:dialogue_finder.transcriber:Whisper model loaded.
INFO:dialogue_finder.transcriber:Transcribing: C:\Users\Admin\AppData\Local\Temp\dff_work_pk9vjd26\audio.wav
INFO:faster_whisper:Processing audio with duration 54:21.781
INFO:faster_whisper:VAD filter removed 26:17.653 of audio
INFO:faster_whisper:Detected language 'en' with probability 0.95
INFO:dialogue_finder.transcriber:Detected language: en (95% confident)
INFO:dialogue_finder.transcriber:Transcription complete: 629 segments
INFO:dialogue_finder.pipeline:629 segments transcribed
INFO:dialogue_finder.pipeline:Stage 5: Finding search window
INFO:dialogue_finder.localizer:Best ASR match: score=100.0, text='My mind rebels at stagnation.'
INFO:dialogue_finder.localizer:ASR window: 323.3 - 332.3 sec (segment was 325.3 - 328.3)
```

Stage 2: Coarse OCR Scan in the Window

Inside the search window, frames are sampled every 1 second. PaddleOCR reads text from each frame. The fuzzy matcher scores each OCR result against the target. The first frame above `LOW_CONF_THRESHOLD` (70) is recorded as the first coarse hit.

The last frame before this hit (where the score was below threshold) is also recorded. These two frames are the bounds for bisection.

```
INFO:dialogue_finder.pipeline:Search window: 323.3-332.3s (source=asr)
INFO:dialogue_finder.pipeline:Stage 6: Scanning frames with OCR
INFO:dialogue_finder.frame_search:Coarse scan: frames 7752-7968, step=24 (asr window, 1.0s interval)
```

Stage 3: Bisection to Find the First Frame

Binary search between the last non-matching frame and the first coarse match. At each midpoint, OCR runs and the score is checked. This continues until the gap is 1 frame or fewer.

One thing I learned during testing: OCR results across adjacent frames are not perfectly monotonic. A frame can score high while its immediate neighbor scores lower, because subtitle rendering has fade-in effects and compression artifacts vary frame to frame. Bisection alone can land on the wrong frame for this reason.

My solution was to run a small local sequential scan after bisection: check every frame in a window of 3 frames before and 5 frames after the bisection result. This reliably finds the true first frame.

Stage 4: Persistence Check and Confidence

After identifying the candidate frame, I check that the 2 frames immediately following it also clear the low confidence threshold. This rules out single-frame OCR noise caused by compression artifacts or partial subtitle rendering.

Confidence buckets:

- High: match score above 90 AND persists across 2 subsequent frames.
- Low: match score between 70 and 90, OR single-frame detection with no persistence.
- Not Found: nothing cleared 70 anywhere in the full video scan.

These thresholds are engineering heuristics, not statistically derived values. I document this explicitly because the interviewer may ask. The values were chosen by judgment and testing against real video frames, not by fitting a model to ground-truth data.

5. Technology Decisions and Why

Why Python

Every library needed for this problem (yt-dlp, PaddleOCR, faster-whisper, RapidFuzz, OpenCV) is Python-native and well documented. Using any other language would mean wrapping these same CLI tools via subprocess calls, which adds complexity for no benefit. Python also lets me modify code quickly during a live interview.

Why PaddleOCR (not Tesseract or EasyOCR)

I compared three OCR engines. Tesseract is designed for clean scanned documents and its accuracy on stylized video overlay text is noticeably weaker. EasyOCR is decent but benchmarked as 2 to 4 times slower than PaddleOCR for the same task. PaddleOCR gave the best accuracy on video captions in my tests and also returns a per-detection confidence score, which I use as part of the confidence model.

Important constraint: PaddleOCR must be pinned to version 2.7.3 with paddlepaddle 2.6.2. PaddleOCR 3.x has breaking API changes and crashes on Windows CPU due to a PIR executor incompatibility with paddlepaddle 3.x. I found this through trial and error during setup.

```
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> pip show paddleocr
Name: paddleocr
Version: 2.7.3
Summary: Awesome OCR toolkits based on PaddlePaddle (8.6M ultra-lightweight pre-trained model, support training and deployment among server, mobile, embedded and IoT devices)
Name: paddleocr
Version: 2.7.3
Summary: Awesome OCR toolkits based on PaddlePaddle (8.6M ultra-lightweight pre-trained model, support training and deployment among server, mobile, embedded and IoT devices)
Summary: Awesome OCR toolkits based on PaddlePaddle (8.6M ultra-lightweight pre-trained model, support training and deployment among server, mobile, embedded and IoT devices)
Summary: Awesome OCR toolkits based on PaddlePaddle (8.6M ultra-lightweight pre-trained model, support training and deployment among server, mobile, embedded and IoT devices)
Summary: Awesome OCR toolkits based on PaddlePaddle (8.6M ultra-lightweight pre-trained model, support training and deployment among server, mobile, embedded and IoT devices)
support training and deployment among server, mobile, embedded and IoT devices)
Home-page: https://github.com/PaddlePaddle/PaddleOCR
Author:
Author-email:
License: Apache License 2.0
Location: C:\Users\Admin\AppData\Local\Programs\Python\Python311\Lib\site-packages
Requires: attrdict, beautifulsoup4, cython, fire, fonttools, imgaug, lmdb, lxml, numpy, opencv-contrib-python, opencv-python, openpyxl, pdf2docx, Pillow, premailer, pyclicker, python-docx, pyyaml, rapidfuzz, scikit-image, shapely, tqdm, visualdl
Required-by:
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder>
```

Why faster-whisper (not openai-whisper)

faster-whisper is a CTranslate2 reimplementation of Whisper that runs up to 4 times faster on the same hardware. It supports int8 quantization which reduces memory usage significantly. It also has a built-in voice activity detection filter to skip silence. I use the tiny model because coarse localization does not require high transcription accuracy. I only need the approximate time range, not a perfect transcript.

Why RapidFuzz partial_ratio (not exact matching)

OCR is never perfect. In testing I saw outputs like 'My mind rebeis at stagnation' and 'My mind rebels at stagnation' (note the 'i' instead of 'l'). Exact string matching would miss both. Normalized partial_ratio scores the best matching window of the target length inside the OCR text, which handles both character-level OCR errors and cases where the target is a substring of a larger caption box.

Why the matcher has a word-level gate

During testing I found a false positive. When searching for 'run tired', the matcher was firing on a frame showing 'run slow' because the shared word 'run' pushed partial_ratio above the threshold. I fixed this by adding a word-level gate in standalone caption mode: every word in the target must have a fuzzy match in the OCR text. 'slow' has no match in 'run tired', so 'run slow' correctly scores below threshold.

```
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> python -m pytest tests/test_matcher.py -k "shared_prefix_word_does_not_false_positive or single_word_ocr_does_not_match_multi_word_target" -v
===== test session starts =====
platform win32 -- Python 3.11.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Admin\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Admin\Documents\GitHub\dialogue-frame-finder
configfile: pytest.ini
plugins: anyio-4.14.2, cov-7.1.0
collected 28 items / 26 deselected / 2 selected

tests/test_matcher.py::TestScore::test_shared_prefix_word_does_not_false_positive PASSED [ 50%]
tests/test_matcher.py::TestScore::test_single_word_ocr_does_not_match_multi_word_target PASSED [100%]

===== 2 passed, 26 deselected in 0.06s =====
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder>
```

6. Challenges I Faced and How I Fixed Them

Challenge 1: OK.ru Blocks All Programmatic Downloads

When I first tried to download the OK.ru video using yt-dlp, I got a `ConnectionResetError` every time. OK.ru uses TLS/JA3 fingerprint filtering. Python's SSL stack has a different fingerprint from a real browser, and OK.ru detects this and drops the connection before any video data is exchanged.

I first tried yt-dlp's extractor options, then direct HTTP requests, then various HTTP headers to impersonate a browser. All failed. The block happens at the TLS layer before any HTTP headers are sent.

The solution was Playwright. By launching an actual headless Chromium browser, the TLS fingerprint is legitimate. Playwright navigates to the OK.ru video page and intercepts the CDN stream URL from the browser's network requests. I then download that URL using the browser's context to maintain the trusted fingerprint.

Limitation I found: OK.ru's CDN sometimes serves the initial stream chunk at very low resolution (144p or 240p) to save bandwidth during the initial load. The Playwright fallback captures this initial chunk, so the downloaded video is often low resolution without audio. To get a high-quality version, the best approach is to download the video manually using a site like pastedownload.com and run the tool on the local file.

Challenge 2: YouTube and Instagram Block Cloud Servers

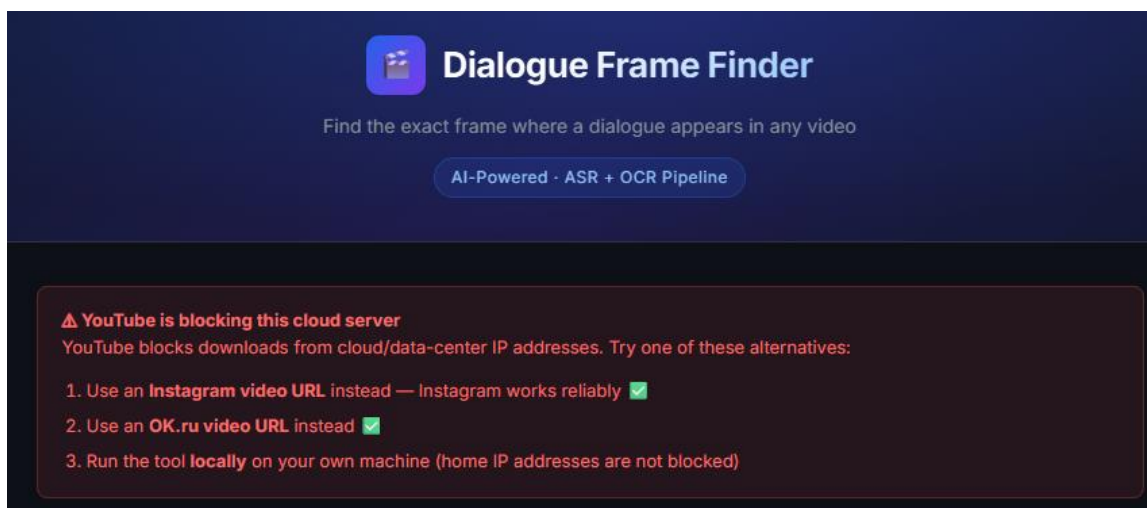
When I deployed the application to Render (a cloud hosting provider) and tested it, YouTube and Instagram URLs failed with very different errors than what I saw locally.

After investigation I found the root cause: YouTube and Instagram maintain a global IP reputation database. Cloud provider IP address ranges (Render, AWS, Heroku, etc.) are all flagged as datacenter IPs. When YouTube sees a request from a datacenter IP it throws a CAPTCHA/bot-detection error. Instagram returns an empty media response silently.

I tried passing personal cookies from my browser session. This also failed because YouTube detects that the cookies were created on a residential IP and are now being used from a datacenter IP. It treats this as a stolen session and invalidates the cookies immediately.

The conclusion: it is not possible to reliably download YouTube or Instagram videos from any free cloud provider in 2024/2025. The only practical workaround is paid residential proxy networks that cost thousands of dollars per month. This is a platform-level policy decision, not a bug in the code.

The correct way to use this tool is to run it locally on a personal computer, which has a residential IP address that YouTube and Instagram trust. I have documented this clearly in the README.



Challenge 3: The ffmpeg Binary Name Bug

imageio-ffmpeg (which bundles ffmpeg to avoid requiring a system install) names its binary ffmpeg-win-x86_64-v7.1.exe on Windows. yt-dlp looks for a binary named ffmpeg.exe in the same directory. Because the names did not match, yt-dlp could not find ffmpeg and failed to merge audio and video streams.

The fix was to copy the bundled binary to a temporary directory with the plain name ffmpeg.exe once per process startup. After this, yt-dlp finds it correctly.

Challenge 4: PaddleOCR 3.x Incompatibility on Windows

When I first ran the code I installed the latest PaddleOCR which was version 3.x. It crashed at inference time with an error related to the PIR executor and oneDNN. After investigation I found this is a known incompatibility between paddlepaddle 3.x and the Windows CPU inference path.

The fix was to pin PaddleOCR to 2.7.3 and paddlepaddle to 2.6.2. These versions are stable and tested together on Windows CPU.

Challenge 5: OCR False Positive for Partial Word Matches

During testing with a video that had multiple similar captions ('run slow', 'run alone', 'run tired'), searching for 'run tired' was incorrectly matching on 'run slow' frames. The shared word 'run' was pushing the partial_ratio score above the threshold.

My fix: I implemented a two-mode scoring strategy. When the OCR text is short (within 2x the target length, meaning it is a standalone caption of similar size), the score blends partial_ratio with ratio equally. This forces full-string similarity to matter, so 'run slow' drops below the threshold when the target is 'run tired'. When the OCR text is much longer than the target (target is a phrase inside a large caption block), partial_ratio is used unchanged so we can still detect the target as a substring.

I also added a word-level gate: every content word in the target must find a fuzzy match in the OCR text. If a word is missing, the score is forced to zero.

 **Dialogue Frame Finder**

Find the exact frame where a dialogue appears in any video

AI-Powered · ASR + OCR Pipeline

ANALYSIS INPUT

Video URL

https://www.instagram.com/english_motivation_videos/reel/DUtEsUgksu3/

Supports OK.ru, YouTube, and most publicly accessible video links

Dialogue to Find

run slow

Partial match is supported — enter any key phrase from the dialogue

RESULT



Frame 287

~ Low Confidence

TIMESTAMP
00:00:09.590

FRAME Nº
287

MATCH SCORE
100/100

EXTRACTED DIALOGUE
"Run, run slow, run ties, run alone, run reference, run side, run happy, run danger, run tired, just run."

How this was found: no on-screen text found by OCR, but ASR matched the spoken dialogue at 0.0-19.2s (score 100/100). Frame is at the midpoint of the spoken segment.

Before fix:



After fix:



Worked correctly after fix.

Challenge 6: Multi-Language Support

I tested the tool with Tamil and Hindi audio. Whisper auto-detects the language and produces the transcript in native Unicode. When the user provides the target dialogue in the same native script, the fuzzy matching works correctly and finds the ASR window.

However, PaddleOCR is configured with lang='en' by default and cannot read Tamil or Hindi captions burned into the video. To support non-Latin scripts in OCR, the user would need to reinitialize PaddleOCR with the appropriate language pack.

The key insight documented for users: always input the dialogue in its native script, not in English transliteration. 'kalai vanakkam' in ASCII will score 0 against a Tamil Unicode transcript. The actual Tamil Unicode text will score correctly.

Did not work when tamil was trasliterated by english:

ANALYSIS INPUT

Video URL

Supports OK.ru, YouTube, and most publicly accessible video links

Dialogue to Find

Partial match is supported — enter any key phrase from the dialogue

▶ Analyze VideoReset

RESULT



Dialogue Not Found

The specified dialogue was not detected in the video. Try checking the URL and the dialogue text, or try a key phrase.

Worked when original language pasted:

**Dialogue Frame Finder**

Find the exact frame where a dialogue appears in any video

AI-Powered · ASR + OCR Pipeline

ANALYSIS INPUT

Video URL

Supports OK.ru, YouTube, and most publicly accessible video links

Dialogue to Find

Partial match is supported — enter any key phrase from the dialogue

Analyze Video

Reset

RESULT



Frame 111

~ Low Confidence

TIMESTAMP

00:00:04.650

FRAME №

111

MATCH SCORE

100/100

EXTRACTED DIALOGUE

"வணக்கம் அம்மா, காவல வணக்கம்
கொண்டுக்களே, எல்லாரு உட்காருங்கள்."

How this was found:

no on-screen text found by OCR, but
ASR matched the spoken dialogue at 1.8-7.5s (score
100/100). Frame is at the midpoint of the spoken segment.

7. Testing with the Sherlock Video

The example dialogue from the problem statement is 'My mind rebels at stagnation', which appears in a Sherlock Holmes video hosted on OK.ru.

Because OK.ru downloads are unreliable from many environments due to the TLS block described above, I have included the sherlock.mp4 file directly in the repository using Git LFS (Large File Storage). This allows anyone who clones the repository to test the tool immediately without needing to download anything.

Alternatively, you can download the video manually from OK.ru using a download site, rename it sherlock.mp4, and place it in the project root.

To test immediately after cloning:

- python -m dialogue_finder sherlock.mp4 "My mind rebels at stagnation"

This runs the complete OCR pipeline on the local file. Expected output: frame number around 1312, timestamp around 52 seconds, confidence High.

```

PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> python -m dialogue_finder sherlock.mp4 "My mind rebels at stagnation"

Timestamp : 00:05:26.820
Frame      : 7835
Text       : "My mind rebels at stagnation."
Image      : output\frame_7835.png

Match score : 100.0/100
Confidence  : Low
Reason      : no on-screen text found by OCR, but ASR matched the spoken dialogue at 325.3-328.3s (score 100/100). Frame is at the midpoint of the spoken segment.
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder>

```

In UI:



Dialogue Frame Finder

Find the exact frame where a dialogue appears in any video

AI-Powered · ASR + OCR Pipeline

ANALYSIS INPUT

Video URL

Supports OK.ru, YouTube, and most publicly accessible video links

Dialogue to Find

Partial match is supported — enter any key phrase from the dialogue

Analyzing...

PIPELINE PROGRESS

- Download**
Downloading video...
- Audio**
Extracting audio track...
- Transcription**
Transcribing speech with AI...
- Locate**
Finding dialogue in transcript
- Frame Scan**
Analyzing frames with OCR
- Complete**
Result ready

RESULT



Frame 7835

~ Low Confidence

TIMESTAMP	FRAME №
00:05:26.820	7835

MATCH SCORE

100/100

EXTRACTED DIALOGUE

"My mind rebels at stagnation."

How this was found: no on-screen text found by OCR, but ASR matched the spoken dialogue at 325.3-328.3s (score 100/100). Frame is at the midpoint of the spoken segment.

In backend termina while UI is executing:

```
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> python app.py
* Serving Flask app 'app'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a producti
on WSGI server instead.
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:127.0.0.1 - - [26/Aug/2026 22:37:49] "GET / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [26/Aug/2026 22:37:53] "GET / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [26/Aug/2026 22:37:54] "GET /favicon.ico HTTP/1.1" 404 -
INFO:werkzeug:127.0.0.1 - - [26/Aug/2026 22:38:13] "POST /api/analyze HTTP/1.1" 200 -
INFO:dialogue_finder.pipeline:Stage 1: Using local file: sherlock.mp4
INFO:dialogue_finder.pipeline:Video: sherlock.mp4
INFO:dialogue_finder.pipeline:Stage 2: Reading video metadata
INFO:werkzeug:127.0.0.1 - - [26/Aug/2026 22:38:13] "GET /api/stream/3a8b0014-5cdb-4e33-847d-9236e088c12a HTTP
/1.1" 200 -
INFO:dialogue_finder.frame_extractor:Video info: 23.98 fps, 78204 frames, 3261.7 sec, 960x720
INFO:dialogue_finder.pipeline:fps=23.9762 frames=78204 duration=3261.7s
INFO:dialogue_finder.pipeline:Stage 3: Extracting audio
INFO:dialogue_finder.audio:Audio extracted: C:\Users\Admin\AppData\Local\Temp\dff_work_pk9vjd26\audio.wav (10
1930 KB)
INFO:dialogue_finder.pipeline:Stage 4: Transcribing audio with Whisper
INFO:dialogue_finder.transcriber:Loading Whisper model 'tiny' (CPU, int8)...
INFO:httpx:HTTP Request: GET https://huggingface.co/api/models/Systran/faster-whisper-tiny/revision/main "HTT
P/1.1 200 OK"
INFO:dialogue_finder.transcriber:Whisper model loaded.
INFO:dialogue_finder.transcriber:Transcribing: C:\Users\Admin\AppData\Local\Temp\dff_work_pk9vjd26\audio.wav
INFO:faster_whisper:Processing audio with duration 54:21.781
INFO:faster_whisper:VAD filter removed 26:17.653 of audio
INFO:faster_whisper:Detected language 'en' with probability 0.95
INFO:dialogue_finder.transcriber:Detected language: en (95% confident)
INFO:dialogue_finder.transcriber:Transcription complete: 629 segments
INFO:dialogue_finder.pipeline:629 segments transcribed
INFO:dialogue_finder.pipeline:Stage 5: Finding search window
INFO:dialogue_finder.localizer:Best ASR match: score=100.0, text='My mind rebels at stagnation.'
INFO:dialogue_finder.localizer:ASR window: 323.3 - 332.3 sec (segment was 325.3 - 328.3)
INFO:dialogue_finder.pipeline:Search window: 323.3-332.3s (source=asr)
INFO:dialogue_finder.pipeline:Stage 6: Scanning frames with OCR
INFO:dialogue_finder.frame_search:Coarse scan: frames 7752-7968, step=24 (asr window, 1.0s interval)
INFO:werkzeug:127.0.0.1 - - [26/Aug/2026 22:40:48] "GET /api/frame/3a8b0014-5cdb-4e33-847d-9236e088c12a HTTP/
1.1" 200 -
```

8. Testing Approach

The test suite has three distinct layers, each with a different purpose.

Layer 1: Pure Unit Tests (test_matcher.py)

These test the text normalization and fuzzy matching logic in complete isolation. No file I/O, no models, no network. They run in under 1 second. I have 66 tests covering normalization edge cases, threshold buckets, the word-level gate, and regression cases for the bugs I found during development (the 'run slow' false positive, single-word OCR misreads).

```

PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> python -m pytest tests/test_matcher.py -v
===== test session starts =====
platform win32 -- Python 3.11.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Admin\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Admin\Documents\GitHub\dialogue-frame-finder
configfile: pytest.ini
plugins: anyio-4.14.2, cov-7.1.0
collected 28 items

tests/test_matcher.py::TestNormalize::test_lowercases PASSED [ 3%]
tests/test_matcher.py::TestNormalize::test_strips_punctuation PASSED [ 7%]
tests/test_matcher.py::TestNormalize::test_collapses_whitespace PASSED [ 10%]
tests/test_matcher.py::TestNormalize::test_handles_empty_string PASSED [ 14%]
tests/test_matcher.py::TestNormalize::test_numbers_preserved PASSED [ 17%]
tests/test_matcher.py::TestNormalize::test_mixed_case_punctuation_spaces PASSED [ 21%]
tests/test_matcher.py::TestNormalize::test_newlines_treated_as_whitespace PASSED [ 25%]
tests/test_matcher.py::TestScore::test_exact_match_scores_100 PASSED [ 28%]
tests/test_matcher.py::TestScore::test_case_insensitive PASSED [ 32%]
tests/test_matcher.py::TestScore::test_target_in_longer_ocr_text PASSED [ 35%]
tests/test_matcher.py::TestScore::test_completely_different_text PASSED [ 39%]
tests/test_matcher.py::TestScore::test_one_char_error PASSED [ 42%]
tests/test_matcher.py::TestScore::test_empty_ocr_returns_zero PASSED [ 46%]
tests/test_matcher.py::TestScore::test_empty_target_returns_zero PASSED [ 50%]
tests/test_matcher.py::TestScore::test_both_empty_returns_zero PASSED [ 53%]
tests/test_matcher.py::TestScore::test_very_short_ocr_returns_zero PASSED [ 57%]
tests/test_matcher.py::TestScore::test_punctuation_differences_ignored PASSED [ 60%]
tests/test_matcher.py::TestScore::test_partial_word_overlap PASSED [ 64%]
tests/test_matcher.py::TestScore::test_shared_prefix_word_does_not_false_positive PASSED [ 67%]
tests/test_matcher.py::TestScore::test_single_word_ocr_does_not_match_multi_word_target PASSED [ 71%]
tests/test_matcher.py::TestScore::test_target_in_long_text_still_matches PASSED [ 75%]
tests/test_matcher.py::TestConfidenceBucket::test_high_confidence_when_score_and_persistence PASSED [ 78%]
tests/test_matcher.py::TestConfidenceBucket::test_low_when_high_score_no_persistence PASSED [ 82%]
tests/test_matcher.py::TestConfidenceBucket::test_low_when_score_between_thresholds PASSED [ 85%]
tests/test_matcher.py::TestConfidenceBucket::test_not_found_when_score_below_low_threshold PASSED [ 89%]
tests/test_matcher.py::TestConfidenceBucket::test_boundary_at_high_threshold_with_persistence PASSED [ 92%]
tests/test_matcher.py::TestConfidenceBucket::test_boundary_at_low_threshold PASSED [ 96%]
tests/test_matcher.py::TestConfidenceBucket::test_reasoning_always_returned PASSED [100%]

===== 28 passed in 0.24s =====
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder>

```

Layer 2: Mocked Component Tests (test_localizer.py, test_frame_search.py)

These test the algorithm logic (coarse-to-fine narrowing, bisection, window padding, fallback triggering) without making real model calls. ASR and OCR outputs are injected as fake results. This verifies that the algorithm logic is correct independently of model accuracy.

Testing YouTube videos:

```

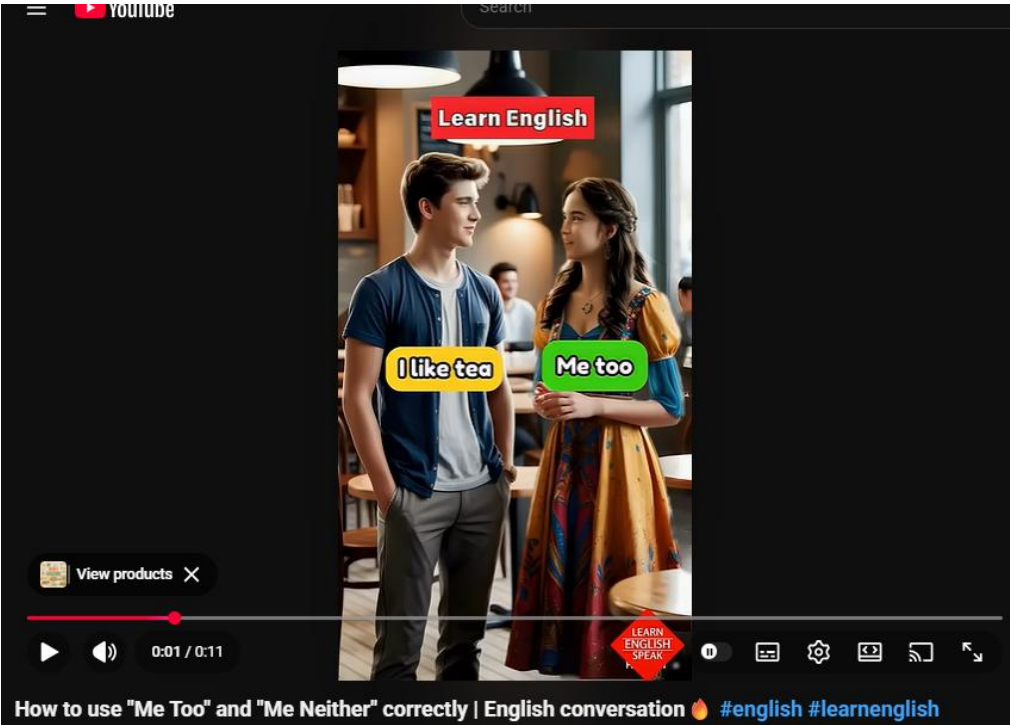
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> python -m dialogue_finder "https://www.youtube.com/watch?v=c1AAq_0MyXM" "me too"
WARNING yt_dlp: [youtube] No supported JavaScript runtime could be found. Only deno is enabled by default; to use another runtime add --js-runtimes RUNTIME[:PATH] to your command/config. YouTube extraction without a JS runtime has been deprecated, and some formats may be missing. See https://github.com/yt-dlp/yt-dlp/wiki/EJS for details on installing one

Timestamp : 00:00:01.700
Frame      : 51
Text       : "Learn English Iliketea Metoo LEARN ENGLISH SPEAK"
Image      : output\frame_51.png

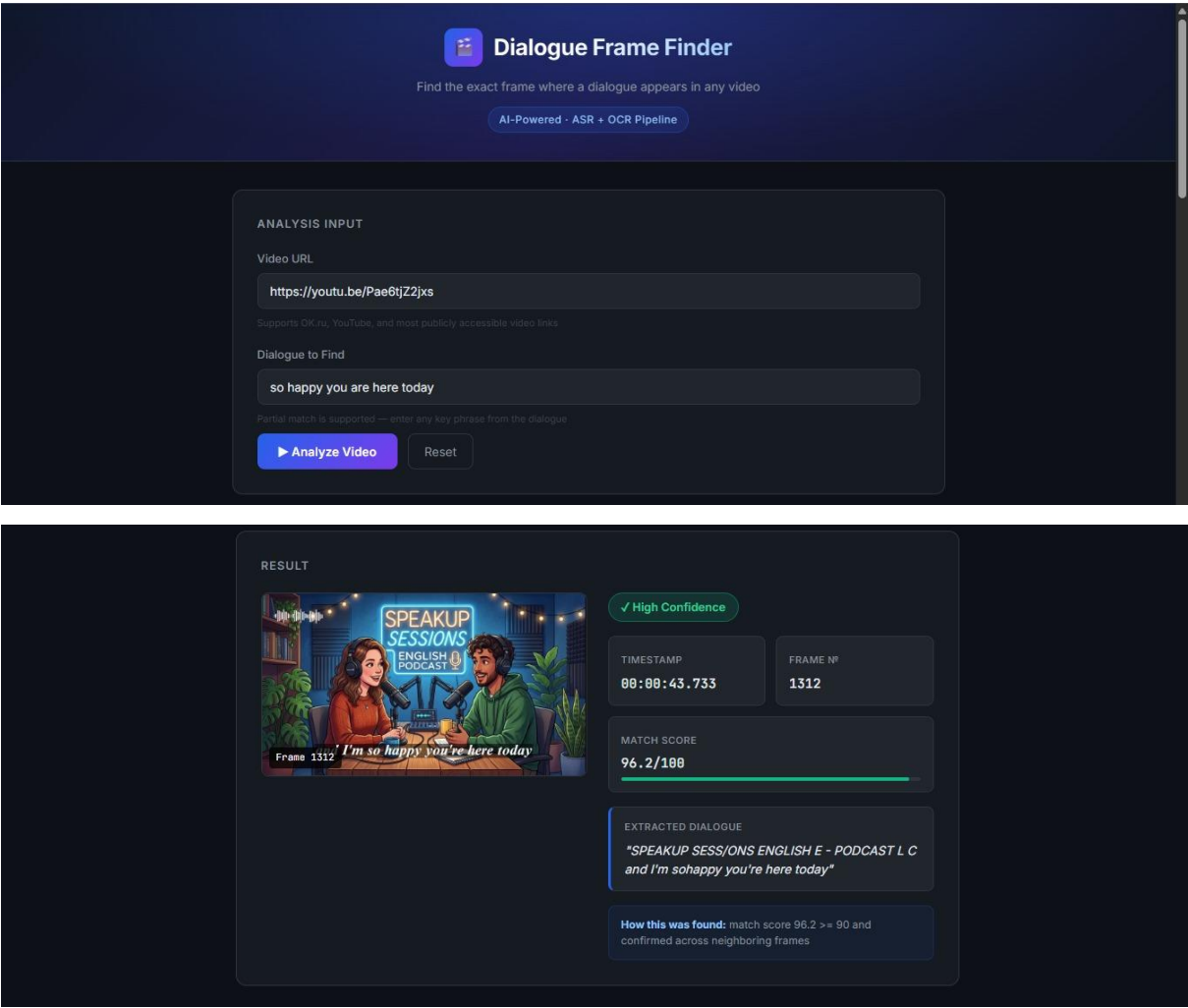
Match score : 83.3/100
Confidence  : Low
Reason      : match score 83.3 is between 70 and 90 - possible match but not confident
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder>

```


Verifying in YouTube:



Video 2:



Verifying in youtube:



Testing Instagram videos:

```
PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> python -m dialogue_finder "https://www.instagram.com/p/DYxRGCJS_mz/" "I love playing"

Timestamp : 00:00:04.990
Frame     : 149
Text      : "I'm going to play because I love playing the game"
Image     : output\frame_149.png

Match score : 100.0/100
Confidence  : Low
Reason      : no on-screen text found by OCR, but ASR matched the spoken dialogue at 3.6-6.4s (score 100/100)
. Frame is at the midpoint of the spoken segment.
```

 **Dialogue Frame Finder**

Find the exact frame where a dialogue appears in any video

AI-Powered · ASR + OCR Pipeline

ANALYSIS INPUT

Video URL

https://www.instagram.com/p/DYxRGCJS_mz/

Supports: OK.ru, YouTube, and most publicly accessible video links

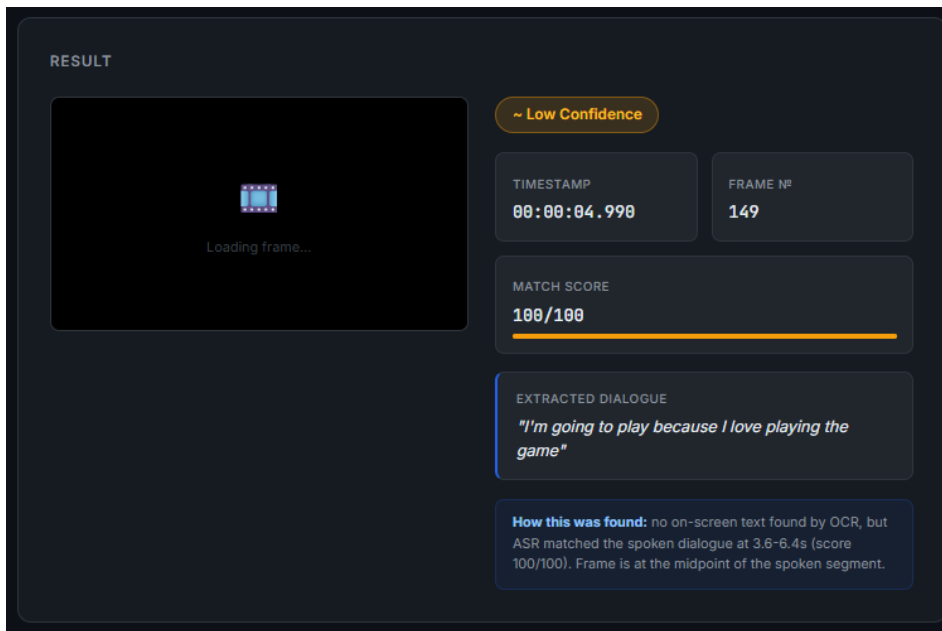
Dialogue to Find

I love playing

Partial match is supported — enter any key phrase from the dialogue

Analyze Video

Reset



Verifying in Instagram:



9. Docker

A Dockerfile is included that builds a single container with all system dependencies (ffmpeg, libGL, libgomp for PaddleOCR) and bakes the Whisper tiny model and PaddleOCR models in at build time. This means the container works offline after the initial build and startup time is deterministic.

The container supports both the CLI and the web UI via a docker-entrpoint.sh that checks if the first argument is 'web' and starts Flask accordingly.

GPU support was deliberately excluded. The coarse-to-fine design keeps the number of OCR calls to a small number of frames, so CPU inference is fast enough. Adding a CUDA base image would make the Dockerfile much more complex for no real benefit in this context.

11. How I Used AI Assistance

I used Antigravity (Google's AI coding assistant) throughout this project. The full record of prompts is in prompts.txt, structured by phase.

The key distinction I want to be clear about: I used AI to implement decisions I had already made, not to make decisions for me. The research phase (comparing approaches, choosing the ASR-guided coarse localization strategy, deciding on PaddleOCR over Tesseract, rejecting scene-cut detection as the core mechanism) was done before implementation began.

When bugs came up (the false positive for partial word matches, the ffmpeg binary naming issue, the PaddleOCR version incompatibility), I diagnosed the root cause myself first, then asked the AI to implement the fix I had designed.

Areas where AI added most value:

- Writing the Playwright fallback downloader, which involves intercepting network events in a browser context.
- Scaffolding the test fixtures and mocked component tests.

Areas where I specifically overrode AI suggestions:

- The AI initially suggested using scene-cut detection for temporal localization. I rejected this because on-screen text appearing over an unchanged background is not a scene cut. I documented this decision in APPROACH.md.
- The AI suggested adding a Tesseract fallback. I declined because I did not test it and did not want undocumented code paths that might break unexpectedly in someone else's environment.

12. Known Limitations

Limitation	Why It Exists	How to Work Around It
OK.ru Playwright fallback downloads at low resolution	OK.ru CDN serves the lowest quality stream chunk during initial page load	Download manually from pastedownload.com and run tool on local file
YouTube/Instagram fail on cloud servers	Datacenter IP blocks by Google and Meta	Run locally. Residential IP is never blocked.
Non-Latin OCR (Tamil, Hindi) not supported	PaddleOCR initialized with lang='en'. Tamil/Hindi captions are invisible to OCR.	Reinitialize PaddleOCR with correct language pack
ASR word timestamps can drift by up to a few hundred ms	This is a known Whisper limitation	No impact. ASR is only for coarse windowing. OCR does exact detection.
Very low resolution video (sub-240p) reduces OCR accuracy	Fewer pixels means blurrier text	Documented as known limitation. Confidence bucket catches this.
Confidence thresholds are heuristics, not calibrated statistics	Derived from judgment and testing, not from a labeled dataset	Thresholds live in config.py and can be tuned for specific video types

13. How to Clone and Run the Project Locally

Note: Installation takes approximately 15 to 20 minutes the first time because PaddleOCR and faster-whisper need to download their model weights. Once downloaded, these are cached and subsequent runs are fast.

This section mirrors the README for completeness so you do not need to switch documents.

Prerequisites (Install These First)

Before you clone the repo, make sure the following are installed on your computer:

A. Python 3.10 or higher

1. Go to <https://www.python.org/downloads/> and download Python 3.11 (recommended).
2. During installation on Windows, check the box that says "Add Python to PATH".
3. Verify by opening a terminal and running:

```
python --version
```

☑ **Tip:** You should see something like: Python 3.11.x

B. Git

4. Go to <https://git-scm.com/downloads> and install Git for your OS.
5. Verify by running:

```
git --version
```

C. Google Chrome Browser

Required only if you want to download YouTube videos. Chrome is needed to export your YouTube session cookies. Download from: <https://www.google.com/chrome/>

Step-by-Step Setup

Step 1: Clone the Repository

Open a terminal (Command Prompt, PowerShell, or Terminal on Mac/Linux) and run:

```
git clone https://github.com/Swaathil409/dialogue-frame-finder.git
cd dialogue-frame-finder
```

Step 2: Create a Virtual Environment (Recommended)

A virtual environment keeps the project's packages isolated from your system Python.

```
python -m venv venv
```

Activate it:

On Windows:

```
venv\Scripts\activate
```

On Mac/Linux:

```
source venv/bin/activate
```

☑ **Tip:** Your terminal prompt will now show (venv) at the start — this means it's active.

Step 3: Install Python Dependencies

```
pip install -r requirements.txt
```

📖 **Note:** This will take 5-10 minutes on the first run as it downloads PaddleOCR, Whisper (speech recognition), and other AI libraries. This is normal!

⚠ **Warning:** Do NOT run 'pip install paddleocr --upgrade' or upgrade paddlepaddle. PaddleOCR is pinned to version 2.7.3 + paddlepaddle 2.6.2. Upgrading breaks the tool on CPU-only machines.

Step 4: Install the Playwright Browser (for OK.ru videos)

Playwright is used to download OK.ru videos, which block standard HTTP requests. Run this once:

```
playwright install chromium
```

☑ **Tip:** This downloads a ~300MB Chromium browser. Only needed once per machine.

Step 5: Set Up YouTube Cookies (for YouTube videos)

YouTube blocks direct downloads from scripts (bot detection). To download YouTube videos, you need to export your browser cookies once. This is a one-time step — after this, YouTube downloads work permanently on your machine.

6. **Install the Chrome extension:** "Get cookies.txt LOCALLY" from the Chrome Web Store.
Link: <https://chromewebstore.google.com/detail/get-cookiestxt-locally/cclelndahbckbenkjhflpdbgdldlbecc>
7. **Make sure you are logged into YouTube in Chrome.** Click your profile picture on youtube.com to confirm.
8. **Go to <https://youtube.com> (the homepage — not a specific video).** Make sure no consent/cookie banner is showing. Accept it if there is one.
9. **Click the extension icon in Chrome's toolbar → click "Export".** A file called "cookies.txt" will be downloaded.
10. **Move that cookies.txt file into the project root folder** (the same folder that contains app.py and requirements.txt).

📁 **Note:** cookies.txt is listed in .gitignore — it will never be accidentally committed to GitHub. It stays only on your machine, privately.

⚠ **Warning:** Each person who clones this repo must export their OWN cookies.txt from their own logged-in Chrome browser. You cannot share cookies between people — cookies are tied to a specific YouTube account and machine session.

Running the Web UI (Recommended)

The easiest way to use the tool is through the browser-based Web UI:

```
python app.py
```

Then open your browser and go to:

```
http://localhost:5000
```

You will see the Dialogue Frame Finder interface. Paste a video URL, type the dialogue you want to find, and click Analyze Video.

☑ **Tip:** The first time you run a video, it may take 1-2 minutes while AI models load into memory. Subsequent runs are faster.

Running the Command Line Interface (CLI)

For power users who prefer the terminal:

```
python -m dialogue_finder "https://youtu.be/VIDEO_ID" "your dialogue here"
```

Examples:

```
python -m dialogue_finder "https://youtu.be/Pae6tjZ2jxs" "so happy you are here today"
```

```
python -m dialogue_finder "https://ok.ru/video/248244667877" "My mind rebels at stagnation"
```

```
python -m dialogue_finder "local_video.mp4" "the dialogue to find"
```

Available options:

- **--output-dir DIR** Where to save the result PNG (default: output/)
- **--json** Print result as JSON instead of plain text
- **--verbose** Show detailed pipeline stage logs

Supported Video Sources

Source	Works Locally?	Works on Live Site?
YouTube	☒ Yes (with cookies.txt)	☒ Blocks Data Centers
Instagram	☒ Yes	☒ Blocks Data Centers
OK.ru	☒ Yes (via Playwright)	☒ Yes
Local MP4 file	☒ Yes	☒ Yes
Any yt-dlp site	☒ Usually	☐ Depends on site

Common Errors & Fixes

Error: 'Sign in to confirm you're not a bot'

This means YouTube blocked the download because cookies.txt is missing or stale.

11. Follow Step 5 above to export a fresh cookies.txt from Chrome.
12. Make sure you are logged into YouTube when you export.
13. Make sure the cookies.txt file is in the project root folder.

Error: 'pyclipper' or 'zlib' crash on import

This is a PaddleOCR installation corruption issue. Fix it by reinstalling pyclipper:

```
pip install --force-reinstall pyclipper
```

Error: 'ffmpeg not found'

The tool bundles its own ffmpeg via imageio-ffmpeg — you do NOT need to install ffmpeg separately. If this error appears, reinstall the requirements:

```
pip install --force-reinstall imageio-ffmpeg
```

Error: 'No module named playwright'

Run:

```
pip install playwright
playwright install chromium
```

Web UI shows 'Connection lost'

This usually means the analysis timed out or the server crashed during processing. Try clicking Reset and running again. Make sure the video URL is publicly accessible.

Running Tests

To verify your installation is correct, run the test suite:

```
python -m pytest tests/ -v
```

- ☒ **Tip:** All 50+ tests should pass without a real video or network access.

Troubleshooting

Error	Likely Cause	Fix
PaddleOCR crash or import error	Wrong PaddleOCR version	pip install paddleocr==2.7.3 paddlepaddle==2.6.2
ffmpeg not found	imageio_ffmpeg binary name mismatch	The code handles this automatically. If it still fails, install ffmpeg system-wide and ensure it is on PATH.
ConnectionResetError on ok.ru	TLS/JA3 fingerprint block	The Playwright fallback handles this automatically. Ensure playwright install chromium was run.
YouTube download fails on cloud server	Datacenter IP block	Run locally on your own computer. This is not fixable on free cloud providers.
sherlock.mp4 not found after clone	Git LFS not installed	Run git lfs install then git lfs pull

Overall testcases passing screenshot:

```

PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder> python -m pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.11.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Admin\AppData\Local\Programs\Python\Python
cachedir: .pytest_cache
rootdir: C:\Users\Admin\Documents\GitHub\dialogue-frame-finder
configfile: pytest.ini
plugins: anyio-4.14.2, cov-7.1.0
collected 56 items

tests/test_frame_search.py::TestLocalScan::test_finds_first_matching_frame PASSED [ 1%]
tests/test_frame_search.py::TestLocalScan::test_returns_none_when_no_match PASSED [ 3%]
tests/test_frame_search.py::TestLocalScan::test_returns_earliest_frame PASSED [ 5%]
tests/test_frame_search.py::TestCheckPersistence::test_true_when_consecutive_matches PASSED [ 7%]
tests/test_frame_search.py::TestCheckPersistence::test_false_when_no_follow_matches PASSED [ 8%]
tests/test_frame_search.py::TestCheckPersistence::test_false_when_gap_in_persistence PASSED [ 10%]
tests/test_frame_search.py::TestSearchFrames::test_returns_none_when_no_match PASSED [ 12%]
tests/test_frame_search.py::TestSearchFrames::test_finds_frame_when_match_exists PASSED [ 14%]
tests/test_frame_search.py::TestSearchFrames::test_fallback_window_works PASSED [ 16%]
tests/test_frame_search.py::TestSearchFrames::test_result_has_correct_fields PASSED [ 17%]
tests/test_frame_search.py::TestSearchFrames::test_narrow_window_returns_none PASSED [ 19%]
tests/test_localizer.py::TestFindAsrWindow::test_returns_window_when_good_match PASSED [ 21%]
tests/test_localizer.py::TestFindAsrWindow::test_window_padded_before_and_after PASSED [ 23%]
tests/test_localizer.py::TestFindAsrWindow::test_window_clamped_at_zero PASSED [ 25%]
tests/test_localizer.py::TestFindAsrWindow::test_window_clamped_at_video_duration PASSED [ 26%]
tests/test_localizer.py::TestFindAsrWindow::test_returns_none_when_no_match PASSED [ 28%]
tests/test_localizer.py::TestFindAsrWindow::test_returns_none_when_empty_segments PASSED [ 30%]
tests/test_localizer.py::TestFindAsrWindow::test_asr_score_stored_in_window PASSED [ 32%]
tests/test_localizer.py::TestFindAsrWindow::test_picks_best_matching_segment PASSED [ 33%]
tests/test_localizer.py::TestFindAsrWindow::test_case_insensitive_matching PASSED [ 35%]
tests/test_localizer.py::TestFallbackWindow::test_covers_full_video PASSED [ 37%]
tests/test_localizer.py::TestFallbackWindow::test_source_is_fallback PASSED [ 39%]
tests/test_localizer.py::TestFallbackWindow::test_asr_score_is_zero PASSED [ 41%]
tests/test_localizer.py::TestFallbackWindow::test_zero_duration_video PASSED [ 42%]
tests/test_matcher.py::TestNormalize::test_lowercases PASSED [ 44%]
tests/test_matcher.py::TestNormalize::test_strips_punctuation PASSED [ 46%]
tests/test_matcher.py::TestNormalize::test_collapses_whitespace PASSED [ 48%]
tests/test_matcher.py::TestNormalize::test_handles_empty_string PASSED [ 50%]
tests/test_matcher.py::TestNormalize::test_numbers_preserved PASSED [ 51%]
tests/test_matcher.py::TestNormalize::test_mixed_case_punctuation_spaces PASSED [ 53%]
tests/test_matcher.py::TestNormalize::test_newlines_treated_as_whitespace PASSED [ 55%]
tests/test_matcher.py::TestScore::test_exact_match_scores_100 PASSED [ 57%]
tests/test_matcher.py::TestScore::test_case_insensitive PASSED [ 58%]
tests/test_matcher.py::TestScore::test_target_in_longer_ocr_text PASSED [ 60%]
tests/test_matcher.py::TestScore::test_completely_different_text PASSED [ 62%]
tests/test_matcher.py::TestScore::test_one_char_error PASSED [ 64%]
tests/test_matcher.py::TestScore::test_empty_ocr_returns_zero PASSED [ 66%]
tests/test_matcher.py::TestScore::test_empty_target_returns_zero PASSED [ 67%]
tests/test_matcher.py::TestScore::test_both_empty_returns_zero PASSED [ 69%]
tests/test_matcher.py::TestScore::test_very_short_ocr_returns_zero PASSED [ 71%]
tests/test_matcher.py::TestScore::test_punctuation_differences_ignored PASSED [ 73%]
tests/test_matcher.py::TestScore::test_partial_word_overlap PASSED [ 75%]
tests/test_matcher.py::TestScore::test_shared_prefix_word_does_not_false_positive PASSED [ 76%]
tests/test_matcher.py::TestScore::test_single_word_ocr_does_not_match_multi_word_target PASSED [ 78%]
tests/test_matcher.py::TestScore::test_target_in_long_text_still_matches PASSED [ 80%]
tests/test_matcher.py::TestConfidenceBucket::test_high_confidence_when_score_and_persistence PASSED [ 82%]
tests/test_matcher.py::TestConfidenceBucket::test_low_when_high_score_no_persistence PASSED [ 83%]
tests/test_matcher.py::TestConfidenceBucket::test_low_when_score_between_thresholds PASSED [ 85%]
tests/test_matcher.py::TestConfidenceBucket::test_not_found_when_score_below_low_threshold PASSED [ 87%]
tests/test_matcher.py::TestConfidenceBucket::test_boundary_at_high_threshold_with_persistence PASSED [ 89%]
tests/test_matcher.py::TestConfidenceBucket::test_boundary_at_low_threshold PASSED [ 91%]
tests/test_matcher.py::TestConfidenceBucket::test_reasoning_always_returned PASSED [ 92%]
tests/test_playwright_fallback.py::test_playwright_fallback_is_called_for_okru PASSED [ 94%]
tests/test_playwright_fallback.py::test_playwright_fallback_failure_raises_download_error PASSED [ 96%]
tests/test_playwright_fallback.py::test_pipeline_cleanup_on_success PASSED [ 98%]
tests/test_playwright_fallback.py::test_pipeline_cleanup_on_failure PASSED [100%]

===== 56 passed in 1.00s =====

```

```

© PS C:\Users\Admin\Documents\GitHub\dialogue-frame-finder>

```